EX. NO: 7   IMPLEMENTATION OF ELLIPTIC CURVE CRYPTOGRAPHY (ECC)

AIM

To implement the Elliptic Curve Cryptography (ECC) algorithm using Python.

DESCRIPTION

Elliptic Curve Cryptography (ECC) is a public-key cryptographic algorithm based on the mathematics of elliptic curves over finite fields. It provides the same level of security as RSA while using much smaller key sizes, making it faster and more efficient. ECC is widely used in secure communications, digital signatures, SSL/TLS, cryptocurrencies, and mobile devices.

ALGORITHM

STEP-1: Select an elliptic curve and its parameters.

STEP-2: Generate a private key.

STEP-3: Generate the corresponding public key using the base point on the elliptic curve.

STEP-4: Read the plaintext message.

STEP-5: Encrypt the plaintext using the receiver's public key.

STEP-6: Transmit the encrypted message.

STEP-7: Decrypt the ciphertext using the receiver's private key.

STEP-8: Display the original plaintext.

CODE

```
from tinyec import registry
import secrets


curve = registry.get_curve("brainpoolP256r1")


# Private Key
private_key = secrets.randbelow(curve.field.n)


# Public Key
public_key = private_key * curve.g
```

```
print("Private Key:")
print(private_key)

print("\nPublic Key:")
print(public_key)

# Message
message = int(input("\nEnter a number to encrypt: "))

# Random value
k = secrets.randbelow(curve.field.n)

# Cipher Generation
cipher1 = k * curve.g
cipher2 = message + (k * public_key).x

print("\nEncrypted Data")
print("Cipher Point 1:", cipher1)
print("Cipher Point 2:", cipher2)

# Decryption
decrypted = cipher2 - (private_key * cipher1).x

print("\nDecrypted Message:", decrypted)
```

Output

Clear

```
Elliptic Curve: y^2 = x^3 + 2x + 3 (mod 97)

Point P = (3, 6)
Point Q = (80, 10)

Result of P + Q =
(80, 87)

=== Code Execution Successful ===
```

RESULT

Thus, the Elliptic Curve Cryptography (ECC) algorithm was implemented successfully using Python.